Certified Kubernetes Administrator (CKA): Kubernetes Foundations

Kubernetes Architecture and Lab Setup

Tim Warner
Principal Author, Pluralsight

TechTrainerTim.com

Four framing questions

What's the role of each Kubernetes control plane and worker node component?

What are the differences between the CRI, CNI, and CSI extension interfaces?

How do I create a multi-node kind cluster and verify cluster health?

What's the comparison between my local Kubernetes lab environment and the CKA exam's PSI Bridge desktop environment?

Checking in with Globomantics

“The VP approved our Kubernetes migration last week, and I’m our first dedicated cluster admin. She said, ‘Before you deploy a single container, I need you to understand every component running inside this cluster -- because when something breaks at 2 AM, architecture knowledge is the difference between a five-minute fix and a five-hour panic.’”

Devon W., Job Role, Globomantics Robotics Company

What is the role of each control plane and worker node component?

About Kubernetes

From Greek, meaning a ship “helmsman” or “pilot”

Open-source platform for orchestrating containerized applications at scale in any environment (cloud and/or local)

Released by Google in 2014; now maintained by Cloud Native Computing Foundation (CNCF)

Provides automated deployment, scaling, networking, and self-healing for container workloads

Configuration is declarative, not imperative

Kubernetes cluster architecture

Control plane

- API server (single entry point)
- etcd (all cluster state)
- scheduler (pod placement)
- controller manager (reconciliation loops)

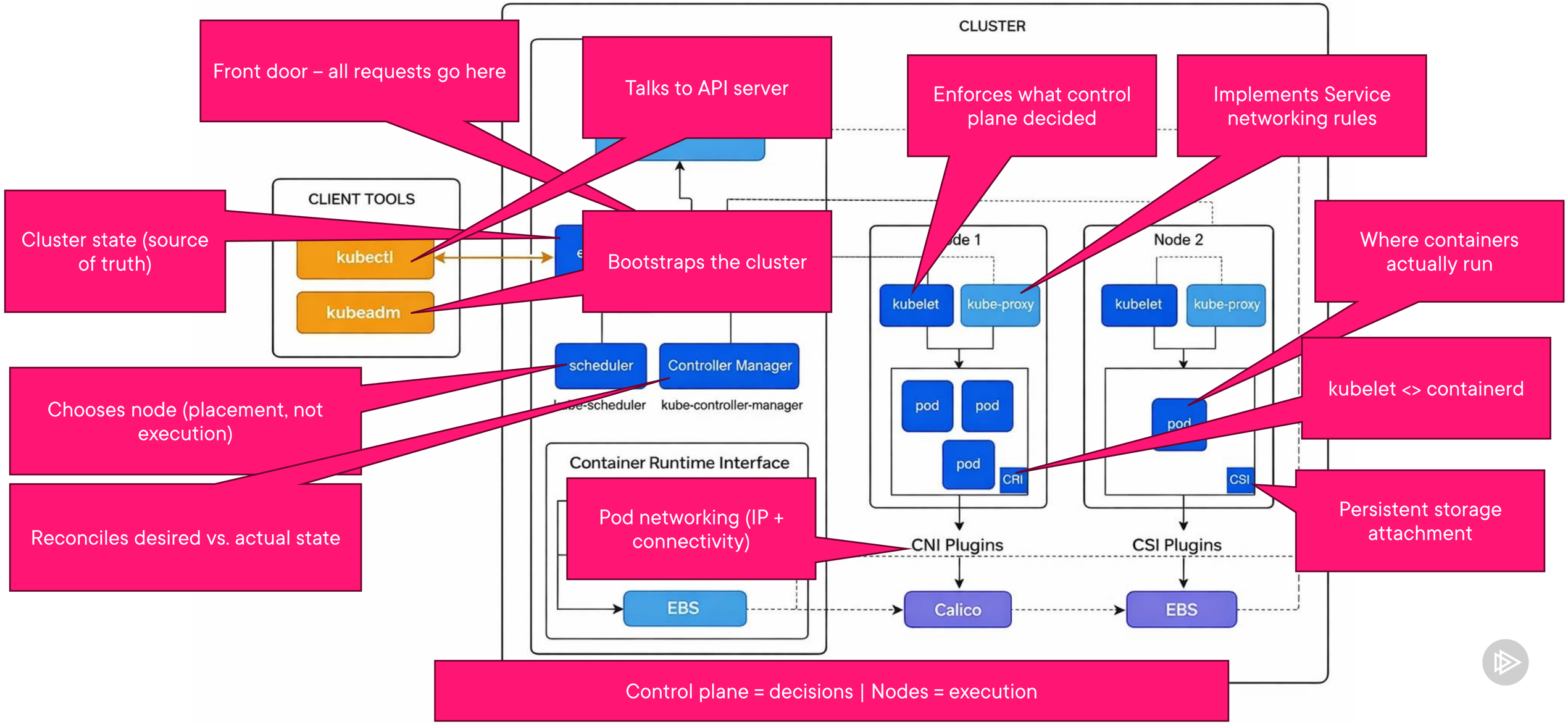
Worker nodes:

- kubelet (pod lifecycle),
- kube-proxy (Service network rules)
- container runtime (containerd)

First principles:

- Every request flows through the API server -- no exceptions
- etcd is the source of truth; lose it without backup and you lose the cluster

Kubernetes cluster architecture: Your mental model



Kubernetes cluster architecture: Your mental model

Every command you run hits the API server – it’s the control plane front door.

Exam tip

What are the CRI, CNI, and CSI extension interfaces?

Kubernetes extension interfaces

CRI

Container Runtime Interface

How the kubelet talks to the container runtime

Plugin: containerd

Replaced Docker shim in v1.24

CNI

Container Network Interface

How pods get IP addresses and network connectivity

Plugins: Flannel, Calico

One pod = one IP

CSI

Container Storage Interface

How persistent storage is attached

Plugins: EBS, Azure Storage, local-path

Decouples storage from core

These are plugin boundaries, not implementations -- Kubernetes defines the interface, vendors supply the plugin. When something breaks, it's usually one of these boundaries.

Exam tip

CNCF projects

CLOUD NATIVE COMPUTING FOUNDATION

About Projects Training Community Blog & News Join

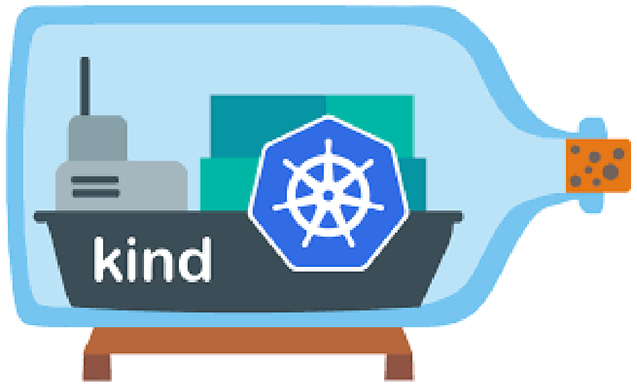
Our Graduated Projects

VIEW ON CNCF LANDSCAPE

argo Continuous Integration & Delivery	CERT MANAGER Security & Compliance	cilium Cloud Native Network	cloudevents Streaming & Messaging	containerd Container Runtime
CoreDNS Coordination & Service Discovery	cri-o Container Runtime	Crossplane Scheduling & Orchestration	FS CubeFS Cloud Native Storage	dapr Application Definition & Image Build
Dragonfly Container Registry	envoy Service Proxy	etcd Coordination & Service Discovery	Falco Security & Compliance	fluentd Observability
flux Continuous Integration & Delivery	HARBOR Container Registry	HELM Application Definition & Image Build	in-toto Security & Compliance	Istio Service Mesh

How do I create a kind cluster and verify cluster health?

About kind



kind = Kubernetes IN Docker

Each node (control plane + workers) is a container, so clusters are lightweight and disposable

Uses kubeadm internally, so behavior matches real CKA exam clusters

Supports multi-node clusters on your laptop

Is fast (<30s), reproducible, and requires no cloud account

Building a 3-node kind cluster

kind cluster config

- `kind: Cluster`
- `nodes: 1 control-plane + 2 workers`
- `extraPortMappings: 80, 443, 30000`
- Reusable lab config
- `kind create cluster --config cka-lab.yaml --name cka-lab`
- Cluster ready in ~25 seconds

kubectl quick setup

- `kubectl cluster-info`
- `kubectl get nodes -o wide`
- `alias k=kubectl`
- `complete -o default -F __start_kubectl k`
- `kubectl config get-contexts`
- `kubectl -n kube-system get pods -o wide`
- `kubectl get svc -n kube-system`

CoreDNS and your kind lab environment

Cluster DNS service for all service discovery

Every service gets a DNS name: `service-name.namespace.svc.cluster.local`

Runs as Deployment + ClusterIP in kube-system

Verify with: `nslookup kubernetes.default`

Minimum viable cluster health checks

`kubectl get nodes -o wide`

Node status: all Ready

containerd runtime, unique IPs

`kubectl –n kube-system get pods`

CP components

etcd, scheduler

kube-proxy, CoreDNS

`nslookup kubernetes.default`

DNS check

Resolves to ClusterIP

How does my lab compare to the CKA exam environment?

Your kind lab vs. the PSI Bridge exam desktop environment

Your kind lab

- WSL2 (Ubuntu) or macOS/Linux terminal
- kind cluster (kubead-based bootstrap)
- kubectl + browser (docs access)
- `alias k=kubectl` + bash completion
- Unlimited time, full Internet access
- Ctrl+C / Ctrl+V copy-paste

CKA exam (PSI Bridge)

- XFCE desktop on Ubuntu (PSI Secure Browser)
- kubeadm clusters (same bootstrap model)
- Firefox locked to kubernetes.io/docs + helm.sh/docs
- `alias k=kubectl` + bash completion pre-configured
- 2-hour time limit, single monitor required
- Terminal: `Ctrl+Shift+C/Ctrl+Shift+V`

Set up your terminal

```
# Create a short alias 'k' for kubectl (saves time on every command)
alias k=kubectl

# Usage example:
k get pods         # instead of: kubectl get pods
k get nodes -o wide

# Enable kubectl bash completion for current shell
source <(kubectl completion bash)

# Attach autocomplete to the 'k' alias
complete -o default -F __start_kubectl k

# Usage example:
k get po<TAB>      # expands to: k get pods
k describe no<TAB>  # expands to: k describe nodes
```

PSI Bridge

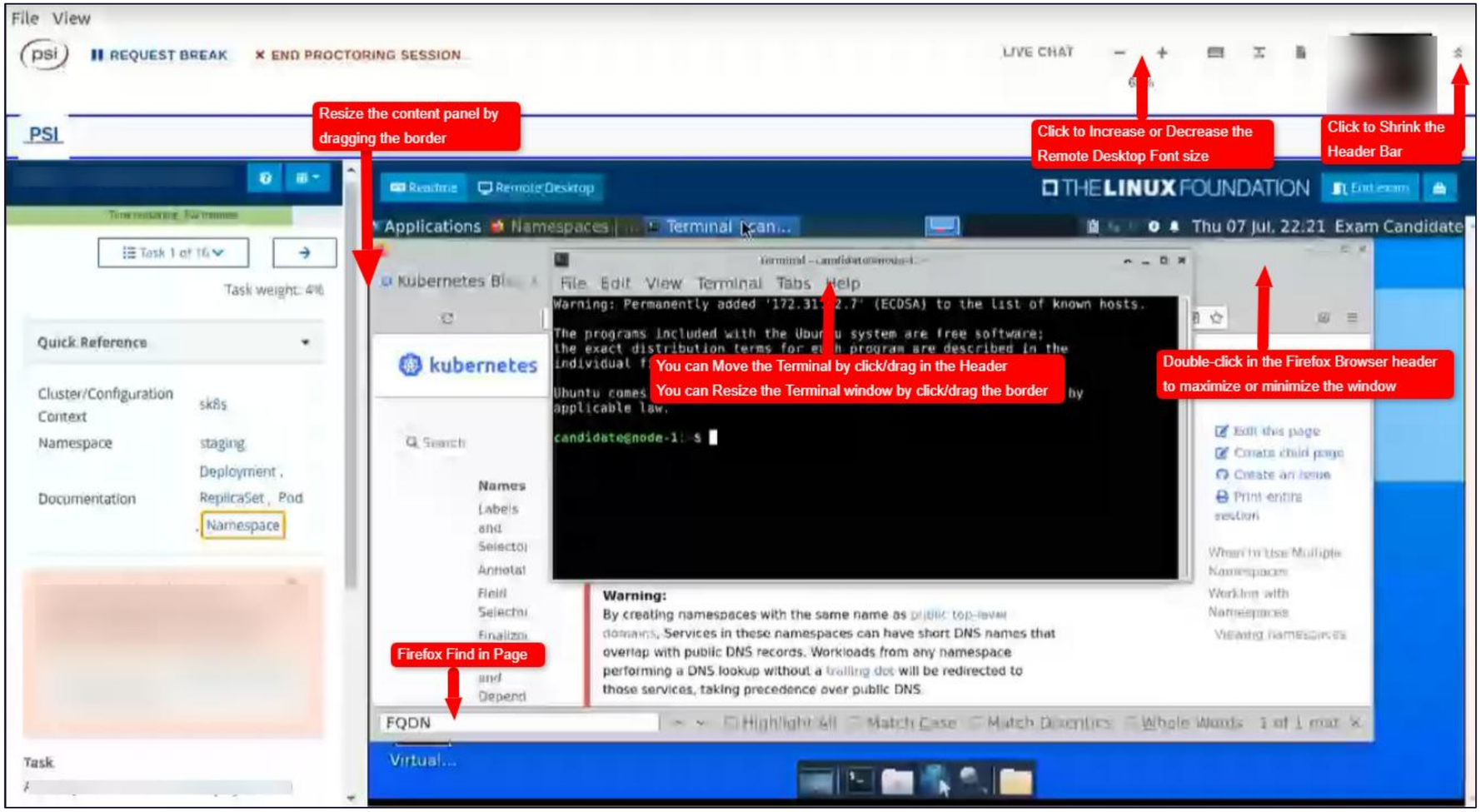


Image Source: **timw.info/bridge**

Demo

Building and exploring the kind cluster

Create the kind cluster from config

Demo

Verify nodes, kube-system pods, and DNS

Demo

Configure exam-ready aliases and compare to PSI Bridge

Demo

Checking out with Globomantics

“Now I understand why I spent my first day on architecture instead of deployments. When our scheduler went sideways last Thursday, I diagnosed it in minutes because I knew exactly which component was responsible.”

Devon W., Job Role, Globomantics Robotics Company

From Globomantics to You

Start with architecture

Map every component in your cluster and know its failure mode

Build your lab today

A kind cluster mirrors the CKA exam and takes only 25 seconds to spin up

Make the k alias muscle memory

Every second saved on typing is a second earned for problem solving

Treat every kubectl command as a diagnostic

Routine get and describe commands teach you how healthy clusters behave